

NN-Adaptive Bayesian Quantum Estimation

Michał Chrzanowski

7 September 2026

Introduction

This report tests reinforcement-learning algorithms that decide when to measure a qubit, so that its frequency is learned as precisely as physically possible. The work implements adaptive Bayesian experimental design for single-qubit Ramsey frequency estimation. A sequential Monte Carlo filter maintains a posterior over the unknown frequency ω , and a policy reads that posterior and picks the interrogation time t for the next shot. The measurement outcome then is used to correct the posterior with particle filter, and the cycle repeats. The policy is trained either by reinforcement learning (TRPO) or by a gradient-free evolutionary search (CEM), following Fiderer, Schuff & Braun, *Neural-network heuristics for adaptive Bayesian quantum estimation*. Two learned policies are built on the same Gymnasium environment, and are evaluated on 10 000 randomly selected ω . TRPO and CEM land in a statistical tie (2.173×10^{-3} vs 2.137×10^{-3} Bayes risk).

Full name	Abbreviation
Sequential Monte Carlo	SMC
Trust-Region Policy Optimisation	TRPO
Cross-Entropy Method	CEM
Effective Sample Size	ESS
Reinforcement Learning	RL

Table of contents

- The experiment
- The physics

- Parameters
 - The methods: TRPO and CEM
 - Training
 - Evaluation
 - Analysis of output data
 - Summary
 - Literature
-

1 The experiment

The system is a single qubit undergoing Ramsey interferometry in the presence of pure dephasing:

1. Prepare $|+\rangle = (|0\rangle + |1\rangle)/\sqrt{2}$.
2. Let it precess freely under $H = \frac{\omega}{2}\sigma_z$ for a time t . This is the control variable - the one thing the experimenter chooses.
3. Apply a $\pi/2$ read-out pulse and measure in the computational basis.
4. Record the binary outcome $d \in \{0, 1\}$.

The qubit is not isolated: transverse noise destroys the coherence $\rho_{01}(t) = \rho_{01}(0) e^{-t/T_2} e^{-i\omega t}$ on a timescale T_2 .

2 The physics

2.1 The likelihood

Solving the Lindblad pure-dephasing master equation and applying the Born rule gives the outcome probability

$$\begin{aligned}
P(d=0 \mid \omega, t) &= e^{-t/T_2} \cos^2\left(\frac{\omega t}{2}\right) + \frac{1 - e^{-t/T_2}}{2} \\
&\equiv \frac{1}{2} [1 + e^{-t/T_2} \cos(\omega t)],
\end{aligned} \tag{1}$$

and $P(d=1) = 1 - P(d=0)$. The two forms are algebraically identical.

The expression reads as a decaying interference fringe:

Table 1: The likelihood.

term	meaning
$\cos^2(\omega t/2)$	the Ramsey fringe - this is what carries information about ω
$v(t) = e^{-t/T_2}$	the visibility, or coherent fraction
$(1 - v)/2$	the fully dephased fraction, which returns a coin flip and tells you nothing

As $t \rightarrow \infty$ the visibility vanishes, $P(d=0) \rightarrow 1/2$, and the measurement becomes pure noise. Decoherence therefore imposes a hard ceiling on how long it is worth interrogating.

Both the simulator and the filter’s likelihood use this same expression, and T_2 is treated as exactly known. The model is well-specified: there is no model mismatch and no nuisance parameter.

2.2 Why long interrogation times stop helping

Fisher information is the right tool for quantifying how much one shot can say about ω . Differentiating the likelihood gives the single-shot (classical) Fisher information

$$I(\omega; t) = \frac{(\partial_\omega P)^2}{P(1-P)} = \frac{t^2 v^2 \sin^2(\omega t)}{1 - v^2 \cos^2(\omega t)}, \quad v = e^{-t/T_2}. \tag{2}$$

Without decoherence ($v \rightarrow 1$) this reduces to $I = t^2$: information grows quadratically in the interrogation time and is independent of ω . With finite T_2 the factor $v^2 = e^{-2t/T_2}$ supplies the envelope $t^2 e^{-2t/T_2}$, which peaks at $t = T_2$. The corresponding bound is $I_{\max} \approx e^{-2} T_2^2 = 0.135 T_2^2$; for $T_2 = 10$ the actual single-shot optimum sits at $t^* \approx 8.4$ -9.5, weakly ω -dependent. For $t \gg T_2$ the envelope decays exponentially and $I \rightarrow 0$: a fully dephased measurement returns a fair coin and carries no information. This is the paper’s observation that “times which exceed T_2 tend to yield no information, which explains why the Bayes risk saturates”.

Cramér–Rao therefore bounds any *fixed* schedule: with N shots, $\text{Var}(\omega) \geq 1/\sum_k I(\omega; t_k)$. Since per-shot information is bounded above, the only way to keep improving is to hold the per-shot information near its $t \approx T_2$ ceiling - which a policy may do only once the posterior has narrowed enough that $t \approx T_2$ no longer aliases. This is the sanity check Fisher information provides; it is not the objective.

2.3 The competing constraint: phase ambiguity

Fisher information is a *local* quantity, and maximising it is the wrong move while the prior is still broad. The likelihood depends on ω only through $\cos(\omega t)$, which is periodic. Over the prior support $\Delta\omega = 1$ the fringe wraps once $t > 2\pi \approx 6.28$: several well-separated ω then produce the same phase. The posterior goes multimodal, and the filter cannot resolve the aliases.

A good policy must therefore trade off:

- small t - unambiguous, but low information per shot ($I \sim t^2$).
- $t \approx T_2$ - maximum information per shot, but aliases a broad prior.
- $t \gg T_2$ - decohered, zero information, strictly wasted.

The resolution is to grow t *as the posterior narrows*, so the phase spread across the posterior, $\sqrt{\text{Var}(\omega)} t$, stays near one radian. This is the rule $t_k = \text{Var}(\omega \mid D_{k-1})^{-1/2}$ - the inverse posterior standard deviation - shown by Ferrie et al. to be optimal in the greedy sense in the asymptotic limit $N \rightarrow \infty$ for the no-decoherence problem. It predicts *monotone growth* of the interrogation time as the posterior narrows, and it is what a learned policy is expected to discover.

2.4 The figure of merit: Bayes risk

This is a Bayesian estimation problem, and the quantity everything is judged by is the Bayes risk - Fiderer Eq. (1):

$$r(h \mid p(\theta)) = \mathbb{E}_{D_k} [\text{tr Cov}(\theta \mid D_k)], \quad (3)$$

where h is the experiment-design heuristic, $D_k = (d_1, \dots, d_k)$ the measurement record, and smaller is better. It is the risk of the quadratic loss $L(\hat{\theta}_k, \theta) = \|\hat{\theta}_k - \theta\|_2^2$ under the Bayes estimator $\hat{\theta}_k(D_k) = \mathbb{E}[\theta \mid D_k]$. That is, the posterior mean, which is exactly what this implementation reports as `posterior_mean`.

The equivalence to the posterior variance is the law of total expectation. With the Bayes estimator $\hat{\theta}_k(D_k) = \mathbb{E}[\theta \mid D_k]$,

$$r = \mathbb{E}_\theta \left[\mathbb{E}_{D_k \mid \theta} [\|\hat{\theta}_k - \theta\|_2^2] \right] = \mathbb{E}_{D_k} \left[\mathbb{E}_{\theta \mid D_k} [\|\hat{\theta}_k - \theta\|_2^2] \right] = \mathbb{E}_{D_k} [\text{tr Cov}(\theta \mid D_k)], \quad (4)$$

using Bayes' law to swap the conditioning and the variance-minimising property of the posterior mean. For a single parameter the trace is the scalar posterior variance.

Here the trace is over a 1×1 covariance, and the expectation is averaged over true ω values. So the Bayes risk is precisely the mean final posterior variance reported as `final_variance_mean`. They are the same number. The paper's name for it is the Bayes risk.

The paper is explicit that the *frequentist* route goes "with the Cramér-Rao bound formalism by maximizing the quantum Fisher information". It deliberately takes the Bayesian route instead. Fisher information is a useful sanity check here (above) but it is not the objective.

2.5 The reward

The reward is the reduction in traced posterior covariance at each step - Fiderer Eq. (2):

$$R(D_k) = \text{tr Cov}(\theta \mid D_{k-1}) - \text{tr Cov}(\theta \mid D_k) \\ \xrightarrow{d=1} V_{k-1} - V_k, \quad (5)$$

with $V_k = \sum_i w_i (\omega_i - \bar{\omega})^2$. This is not an arbitrary choice, and it is not a weaker substitute for an information-gain reward. The paper’s reasoning: the negative Bayes risk is the obvious reward but is far too slow to compute inside a training loop. Eq. (2) is cheap in the SMC framework *and* telescopes to it. Eq. (3), for N experiments per episode:

$$\mathbb{E}_{D_N} \left[\sum_{j=1}^N R(D_j) \right] = \text{const} - r(h \mid p(\theta)), \quad \text{const} = \text{tr Cov}(\theta). \quad (6)$$

So maximising the episode return provably minimises the Bayes risk, provided the return is undiscounted ($\gamma = 1$). The paper additionally sets $\text{tr Cov}(\theta)$ to its exact prior value (1/12 for $\text{Uniform}(0, 1)$) as a numerical constant rather than recomputing it from a finite particle sample, so the reward carries no particle-filter noise from the first step.

3 Parameters

A single shared parameter set backs every comparison.

Table 2: What is being estimated.

quantity	value
parameter	ω , a single scalar (the Larmor frequency / detuning)
prior	$\omega \sim \text{Uniform}(0, 1)$
true values	drawn from the same $\text{Uniform}(0, 1)$
coherence	$T_2 = 10$
time	
control	$t \in [0.1, 3000]$
particles	10^4 train / 2×10^3 eval
episode	100 measurements (training) / 125 (evaluation)
reference	10 000 ω
evaluation	

4 The methods: TRPO and CEM

5 SMC with Liu–West

We estimate a single parameter fixed ω chosen once at the start for every run. Because ω never evolves. Ordinary particle filters rely on the parameter *randomly moving* at each step to keep the particles spread out. Since ω is frozen, nothing spreads them so resampling collapses every particle onto a few duplicated values. Liu–West resampling fixes this by adding random noise to each particle while preserving the posterior’s first two moments.

Particle system. We carry N parameter particles $\{\xi^i\}$ with log-weights $\{\log w_k^i\}$. Shapes are $(N, 1)$ / $(N,)$ in the NumPy convention.

Bayes update. Each new observation y_k at step k reweights particles, carried out entirely in the log domain:

$$\log w_k^i \leftarrow \log w_k^i + \log p(y_k \mid \xi^i) - \text{logsumexp}(\cdot). \quad (7)$$

Renormalised every step, so the weights always sum to one, and only the *weights* change: ω is a static parameter, so its particle *locations* are untouched by the update.

Effective sample size. The Kish N_{eff} ,

$$N_{\text{eff}} = \left(\sum_i (w_k^i)^2 \right)^{-1} \in [1, N], \quad (8)$$

measures how concentrated the weights have become. Resampling fires when $N_{\text{eff}} < 0.75N$, an aggressive threshold chosen deliberately, which when tested worked best.

Liu–West kernel resampling ($a = 0.98$). When the threshold is crossed, each particle is shrunk toward the weighted mean $\bar{\xi}$ and jittered with noise:

$$\xi^i \leftarrow a \hat{\xi}^i + (1 - a) \bar{\xi} + \varepsilon^i, \quad \varepsilon^i \sim \mathcal{N}(0, h^2 \Sigma), \quad h^2 = 1 - a^2, \quad (9)$$

where Σ is the covariance of the particles, weighted by w_k^i .

The shrinkage toward the weighted mean exactly cancels the variance added by the jitter,

$$a^2 \Sigma + h^2 \Sigma = (a^2 + (1 - a^2)) \Sigma = \Sigma, \quad (10)$$

so the posterior’s first two moments are preserved. The particles’ locations move during resampling, so weakly supported regions lose their over-representation and strongly supported regions gain it.

subsectionThe control problem The control problem is framed as a Gymnasium environment:

Table 3: The control problem.

observation	posterior mean, posterior variance, and the last 30 interrogation times, i.e. a 32-dimensional continuous vector
action	interrogation time t , a single continuous value between 0.1 and 3000
reward	$V_{k-1} - V_k$
episode	100 measurements during training, 125 during evaluation, one true ω per episode

5.1 The two learned policies

Two learned policies are implemented:

In the paper these are two *stages* of one pipeline - a CEM-trained network seeds TRPO through imitation learning, combined because CEM explores while TRPO optimises. Here they are instead trained from scratch and compared head-to-head, so each is a complete, independent method on the shared SMC filter and environment.

Table 4: Learned policies.

method	optimiser	network
TRPO	trust-region policy optimisation (from sb3-contrib)	a multilayer perceptron with two hidden layers of 256 units, used for both the policy and the value function
CEM	cross-entropy method over the flat 545-dimensional weight vector	one hidden layer of 16 units with tanh activation and a sigmoid-squashed output

The CEM network squashes its output structurally, $t = t_{\min} + (t_{\max} - t_{\min}) \sigma(z)$, so it can never emit an out-of-range time. The TRPO actor is an unbounded Gaussian whose sample is clipped in the environment step.

5.1.1 1. TRPO

The policy reads a 32-dimensional continuous observation, consisting of the posterior mean, the posterior variance and the last 30 interrogation times, and emits an unbounded Gaussian sample of t , which is clipped to $[0.1, 3000]$ in the environment step. Both the policy and the value head are multilayer perceptrons with two hidden layers of 256 units. The optimiser is the trust-region policy optimiser from `sb3-contrib`, with a target KL of 10^{-2} and $\gamma = 0.99$. TRPO is trained from scratch for 10^7 timesteps.

Training is the standard trust-region loop, one update per rollout batch:

1. Collect experience. The agent plays episodes in the adaptive SMC environment. Each episode draws one true frequency, runs 100 measurements, observes the posterior mean, the posterior variance and the last 30 interrogation times, and receives the reward $V_{k-1} - V_k$. 2048 environment steps are gathered per update - the sb3-contrib default, left unoverridden here.

2. Estimate advantages. Generalised advantage estimation with $\gamma = 0.99$ and $\lambda = 0.95$ converts the reward sequence into a normalised advantage $\hat{A}$ for every step, using the value head as the baseline.
3. Fit the value function. The critic takes 10 gradient steps per update, which lowers the variance of $\hat{A}$.
4. Take a trust-region step. The actor maximises the surrogate $\mathbb{E}\left[\frac{\pi_\theta(a|s)}{\pi_{\theta_{\text{old}}}(a|s)} \hat{A}\right]$ subject to $\mathbb{E}[D_{\text{KL}}(\pi_{\theta_{\text{old}}} \parallel \pi_\theta)] \leq 10^{-2}$. The search direction is the natural gradient, obtained by conjugate gradients (at most 15 iterations, damping 0.1). A backtracking line search then shrinks the step by 0.8 until the KL constraint holds and the surrogate improves. If no such step exists, the update is reverted.
5. Repeat. Steps 1-4 run for 10^5 episodes (10^7 timesteps). The shipped policy is the network after the last update.

5.1.2 2. CEM

A gradient-free cross-entropy method over the flat 545-dimensional weight vector of a single-hidden-layer time policy with 16 units, a tanh activation and a sigmoid-squashed output. Population 1000, elite fraction 0.1, initial standard deviation 1.0, with 100-step episodes and 2000 particles.

Training is a generation loop over that weight vector:

1. Initialise the search distribution. The network’s parameters are flattened into a 545-dimensional vector θ , and the search starts from $\mu = 0$ and $\sigma = 1.0$.
2. Sample a population. Draw 1 000 candidates $\theta_i \sim \mathcal{N}(\mu, \sigma^2)$.
3. Score each candidate. Each θ_i is copied into the network, which maps the observation through 16 tanh units and squashes its output with a sigmoid, $t = t_{\text{min}} + (t_{\text{max}} - t_{\text{min}}) \sigma(z)$. The candidate’s return is the total reward of one full episode played at that generation’s true frequency.
4. Select the elite. Rank the 1 000 candidates by return and keep the best 100 (elite fraction 0.1).
5. Refit the distribution. Set μ to the elite mean and σ to the elite standard deviation, so the next generation samples around the better parameters.
6. Repeat. Steps 2-5 run for 100 generations. The returned policy is the final μ , not a sampled population member.

6 Training

TRPO. Training runs for 10^5 episodes of 100 steps, or 10^7 timesteps, with 10 000 particles, policy and value networks of two hidden layers of 256 units each, and a target KL of 10^{-2} . Checkpoints are written every 50 000 timesteps. A partially trained policy can then be evaluated at any point, and a crash costs at most one interval.

TRPO is trained from scratch.

CEM. Training runs for 100 generations at population 1000 and elite fraction 0.1, with 2000 particles and 100-step episodes, optimising the flat 545-dimensional weight vector of the single-hidden-layer time policy of 16 units. This takes about 45 minutes on CPU, roughly 27s per generation.

Logging. Logged per episode (TRPO) or per generation (CEM): mean reward, mean initial variance, mean final variance, mean effective sample size, mean interrogation time and true frequency. Plus maximum reward, mean sigma and the norm of μ for CEM. Parameters capture the full hyperparameter set.

7 Evaluation

Both policies are scored through one shared NumPy reference harness, which runs at float64 and follows one true frequency at a time. The harness is method-agnostic. The SMC filter, the measurement model and the scoring code are identical for TRPO and CEM. The only method-specific part is the function that maps the posterior summary to the next interrogation time t . The procedure below is therefore the evaluation of both policies.

1. Draw the test frequencies. Draw N true frequencies $\omega \sim \text{Uniform}(0, 1)$ from a fixed seed. The draw is independent of the training draw, so these are genuinely held-out frequencies.
2. Initialise the filter. Represent the prior by 2000 particles drawn $\text{Uniform}(0, 1)$ with uniform weights. This is the belief before the first measurement.
3. Run one episode. Repeat the following 125 times, once per measurement:
 - (a) Read the belief. Normalise the particle weights, compute the posterior mean and variance from them. Assemble the observation the policy sees: the mean, the variance, then the last 30 chosen interrogation times (all zero before the first step).
 - (b) Choose the interrogation time. The policy maps that observation to a single t , clipped to $[0.1, 3000]$.
 - (c) Simulate the measurement. Draw the binary outcome $d \in \{0, 1\}$ from $\Pr(d = 0 \mid \omega, t) = e^{-t/T_2} \cos^2(\omega t/2) + (1 - e^{-t/T_2})/2$ with $T_2 = 10$, evaluated at the *true* ω of the episode.

- (d) Update the posterior. Reweight every particle by the likelihood of the observed outcome d at its own ω .
 - (e) Resample if needed. If the effective sample size falls below $0.75 \times 2000 = 1500$, resample the particles with the Liu-West scheme to restore weight diversity before the next step.
4. Score the episode. Record that episode’s final posterior variance - one number per frequency.
 5. Average. Average the N final variances. For this single-parameter, experiment-limited problem the traced posterior covariance averaged over true ω is exactly the final posterior variance, so this average *is* the Bayes risk of Eq. (1). Lower is better.

The validation reported below uses $N = 10\,000$. The repository’s general-purpose evaluator, `pipeline/evaluate.py`, runs the same procedure over a batched Torch rollout with `-n-omegas` defaulting to 2000.

A GPU is not required. The batched rollout costs roughly 4.4 ms per 125-step, 2000-particle episode on CPU, so a 10 000- ω evaluation takes about a minute.

8 Analysis of output data

8.1 Validation over 10 000 ω

Both learned policies were evaluated on the NumPy reference harness over 10 000 held-out frequencies.

Table 5: Validation over 10 000 ω (the N-scripts).

	TRPO	CEM
training	100 002 episodes, 10^7 timesteps, 2 h 24 m	99 generations $\times$ 1000 pop, 54 min
Bayes risk over 10 000 ω	2.173×10^{-3}	2.137×10^{-3}
mean t	4.00	6.67
min / max t	0.10 / 9.85	2.16 / 15.60
final ESS (of 2000)	1960	1904

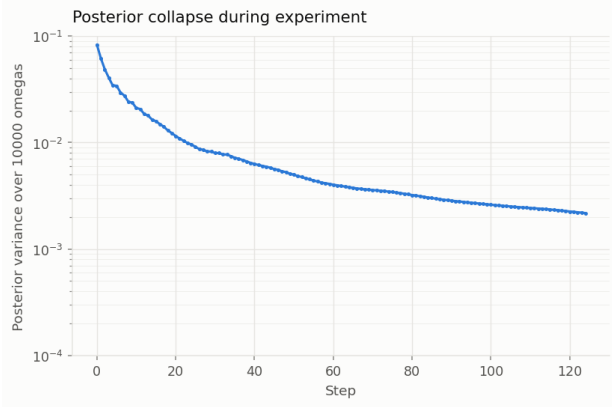
The two methods land in a statistical tie (2.17×10^{-3} vs 2.14×10^{-3} , a 2% difference) despite completely different optimisers and very different chosen times. CEM settles on interrogation times roughly $1.7\times$ longer than TRPO’s, and never goes below $t = 2.16$, where TRPO ranges down to the action floor.

The tie is exactly what the paper reports for this regime. Where the limiting resource is used fully - as here, since all 125 shots are always spent - the CEM and TRPO curves almost

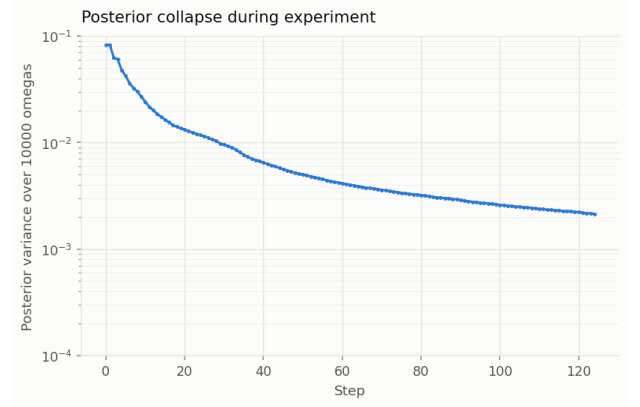
converge. the paper notes TRPO pulls ahead of CEM only when the limiting resource is used only partly. A further practical advantage of TRPO is training speed: per episode, its compute cost is about $10\times$ lower than CEM's, making it the cheaper of the two to train.

8.1.1 Posterior collapse

Mean posterior variance across all 10 000 ω , log scale. Both flatten out well before the 125th measurement. The flattening is the information-starved regime of the Fisher-information analysis: per-shot information is capped at the $t \approx T_2$ ceiling $I_{\max} \approx 0.135 T_2^2$, and both policies hold t near a fixed operating point (≈ 4 for TRPO, ≈ 6.7 for CEM) instead of growing it, so late shots add progressively less. What remains is the decoherence-limited floor set by $T_2 = 10$ and $N = 125$, not a particle-filter failure as ESS stays near 1960 of 2000.



(a) TRPO

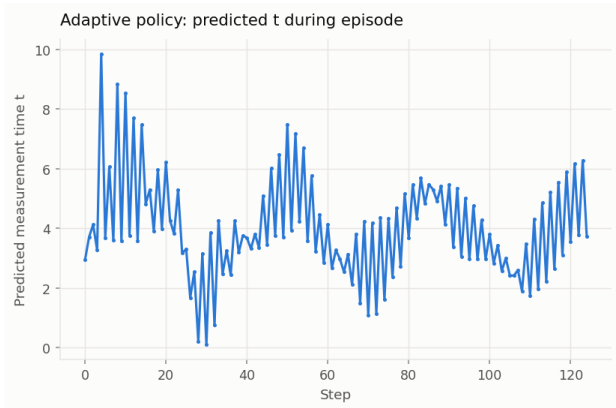


(b) CEM

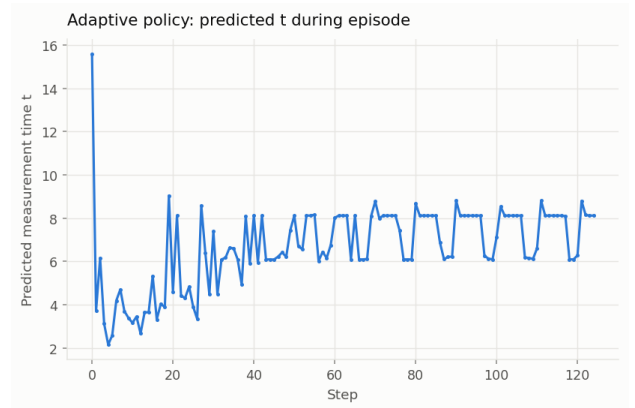
Figure 1: Posterior collapse.

8.1.2 Chosen interrogation times

Bellow are presented interrogation times chosen by the two policies, plotted against the measurement step. The TRPO policy chooses times that are generally shorter than those chosen by the CEM policy as TRPO settles around 04 while CEM settles around 06.7.



(a) TRPO



(b) CEM

Figure 2: Chosen interrogation time versus measurement step, against the coherence time $T_2 = 10$.

9 Summary

This report tested reinforcement-learning algorithms that decide when to measure a qubit, so that its frequency is learned as precisely as physically possible. The work implemented adaptive Bayesian experimental design for single-qubit Ramsey frequency estimation. Sequential Monte Carlo filter that is used to calculate posterior for prior. Trained policy is used to select interrogation time t for the next shot based on posterior. Next measurement outcome corrects the posterior, and the cycle repeats. The policy was trained either by reinforcement learning (TRPO) or by a gradient-free evolutionary search (CEM), following Fiderer, Schuff & Braun, *Neural-network heuristics for adaptive Bayesian quantum estimation*. Two learned policies were built on the same Gymnasium environment and evaluated on 10000 randomly selected ω . TRPO and CEM reach similar performance of $(2.173 \times 10^{-3}$ vs 2.137×10^{-3} Bayes risk. The two policies reach this tie by different strategies: the TRPO policy chooses times that are generally shorter than those chosen by the CEM policy, settling around $t \approx 4$ while CEM settles around $t \approx 6.7$. This tie is consistent with the paper, which reports TRPO outperforming CEM only when there is limited compute resource available.

10 Literature

- Fiderer, Schuff & Braun, *Neural-network heuristics for adaptive Bayesian quantum estimation*, PRX Quantum 2, 020303 (2021) - preprint dated 2020-03-05, whose equation

numbers this report quotes.